

Arrays, Strings, Sets, Maps

Arrays, Lists, Processing Sequences,
Strings and Text Processing, Sets and Maps

JavaScript Data Structures



Arrays

Ordered Lists
for Processing
Sequences



Strings

Text Processing
& Manipulation



Sets

Unique Value
Collections



Maps

Key-Value Pairs
for Associative Data

Svetlin Nakov, PhD

Co-founder @ SoftUni



SoftUni AI

<https://ai.softuni.bg>

Agenda

1. **Arrays** in JS

- Arrays and Operations with Arrays
- Processing Arrays with Arrow Functions

2. **Strings** and Text Processing in JS

- Strings and String Operations, Regular Expressions

3. **Sets** (Sets of Unique Values)

- Using Sets of Elements

4. **Maps** (Dictionaries)

- Key-Value Pairs and Associative Arrays (Maps)



Sli.do Code

#AI-Programming

Join at

slido.com

#AI-Programming



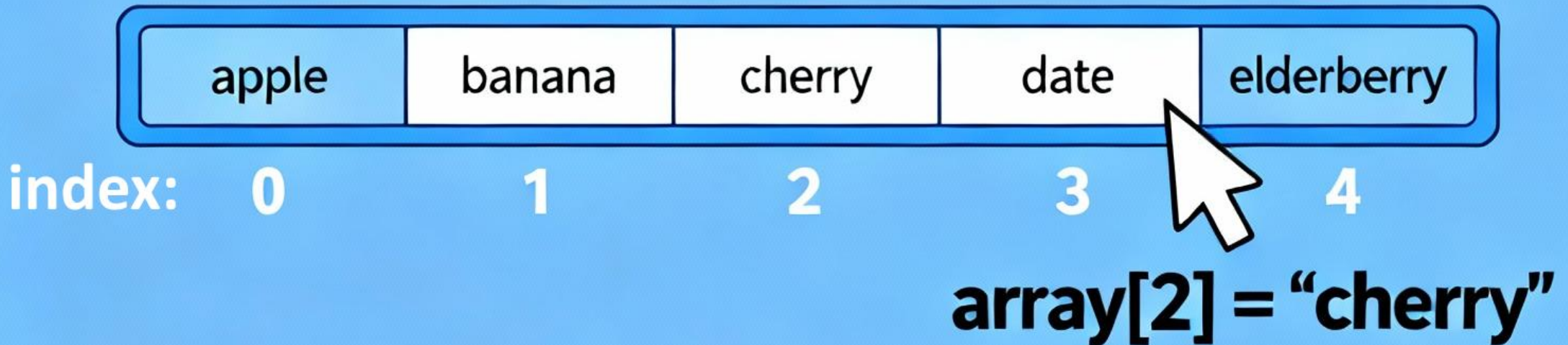
Breaks

20:00 / 21:00



Arrays in JS

Defining Arrays, Accessing Elements



Arrays in JavaScript

- In JavaScript, a **arrays** are indexed lists of elements

```
let arr = [10, 20, 30, 40, 50];
```

- Elements have **indexes** (positions): **0** ... **length-1**



- Accessing elements **by index**:

```
arr[0] = arr[1] + arr[2]; // [50, 20, 30, 40, 50]
```

Arrays in JavaScript

- **Creating** arrays:

```
let arr = ["zero", "one", "two", "three"]; // length = 4
let emptyArr = []; // length = 0
let emptyArrOf5 = new Array(5); // 5 x undefined (length = 5)
let arrOfChars = Array.from("Hi!"); // ['H', 'i', '!']
```

- **Printing** an array:

```
console.log(arr.length); // 4
console.log(arr); // ['zero', 'one', 'two', 'three']
console.log(arr.join(', ')); // zero, one, two, three
```

Problem: Sum First + Last Element

- Write a function to **sum** the **first** with the **last** element of given array of numbers (length ≥ 2)

```
let arr = [20, 30, 40, 50];  
console.log(sumFirstAndLast(arr));  
// 70
```

- Solution:

```
function sumFirstAndLast(arr) {  
    return arr[0] + arr[arr.length - 1];  
}
```


Days of Week – Example

- The **days of week** can be stored in **array of strings**:

```
let days = [  
    "Monday",  
    "Tuesday",  
    "Wednesday",  
    "Thursday",  
    "Friday",  
    "Saturday",  
    "Sunday"  
];
```



Index	Value
days[0]	Monday
days[1]	Tuesday
days[2]	Wednesday
days[3]	Thursday
days[4]	Friday
days[5]	Saturday
days[6]	Sunday

Problem: Day of Week

- Write a function to return the **day of the week** (in English) by given day index [1... 7]
- If the day index is **invalid**, throw an error: "**Invalid day!**"

```
console.log(dayOfWeek(1)); // Monday
console.log(dayOfWeek(5)); // Friday
console.log(dayOfWeek(7)); // Sunday
console.log(dayOfWeek(0)); // Error: Invalid day: 0
console.log(dayOfWeek(8)); // Error: Invalid day: 8
```

Solution: Day of Week

- Keep day names in an **array [0...6]**:

```
function dayOfWeek(day) {  
  if (day < 1 || day > 7)  
    throw new Error("Invalid day: " + day);  
  let days = [ "Monday", "Tuesday", "Wednesday",  
    "Thursday", "Friday", "Saturday", "Sunday" ];  
  return days[day - 1];  
}
```

Judge link: <https://alpha.judge.softuni.org/contests/functions-objects-events/5277>

- Throwing an error** → stops the program execution and prints an error message (unless a **try-catch** is used)

Error Handling in JavaScript

- We can **handle runtime errors** with a **try-catch** block:

```
try {  
    console.log(dayOfWeek(3)); // Wednesday  
    console.log(dayOfWeek(7)); // Sunday  
    console.log(dayOfWeek(0)); // Error: Invalid day: 0  
    console.log("This message will not be printed!");  
}  
catch (err) {  
    console.log(err.message);  
}
```

Iterating over Elements

- **Iterating** over array of elements:

```
let fruits = ['apple', 'banana', 'cherry'];  
for (let i = 0; i < fruits.length; i++)  
    console.log(fruits[i]);
```

```
let dates = [new Date("2025-11-30"),  
    new Date("2026-03-17"), new Date("2026-09-15")];  
for (let date of dates)  
    console.log(date);
```

```
fruits.forEach(f => console.log("Fruit:", f));
```


Problem: Half-Sum

- Write a JS function to take as input an array of **n** numbers, split the array into **two halves** and **compare their sums**

```
let arr = [70, 30, 45, 55];  
halfSum(arr); // left sum (100) = right sum (100)
```

```
let arr = [70, 30, 5, 40, 50];  
halfSum(arr); // left sum (100) > right sum (90)
```

```
let arr = [50, 5, 60];  
halfSum(arr); // left sum (50) < right sum (60)
```

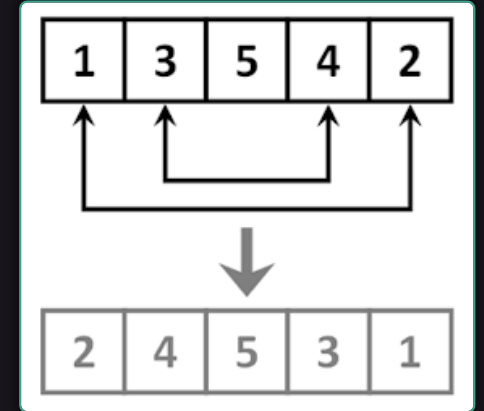
```
let arr = [99999, 0.55555, 1, 777, 2, 99999.55555, -1];  
halfSum(arr); // left sum (100000.55555) = right sum (100000.55555)
```

Solution: Half-Sum

```
function halfSum(arr) {  
    let leftSum = 0, rightSum = 0;  
    for (let l = 0; l < Math.trunc(arr.length / 2); l++)  
        leftSum += arr[l];  
    for (let r = Math.trunc((arr.length + 1) / 2); r < arr.length; r++)  
        rightSum += arr[r];  
    let sign = (leftSum == rightSum) ? "=" :  
        (leftSum > rightSum) ? ">" : "<";  
    console.log(  
        `left sum (${leftSum}) ${sign} right sum (${rightSum})`  
    );  
}
```

Multi-Assignment with Arrays

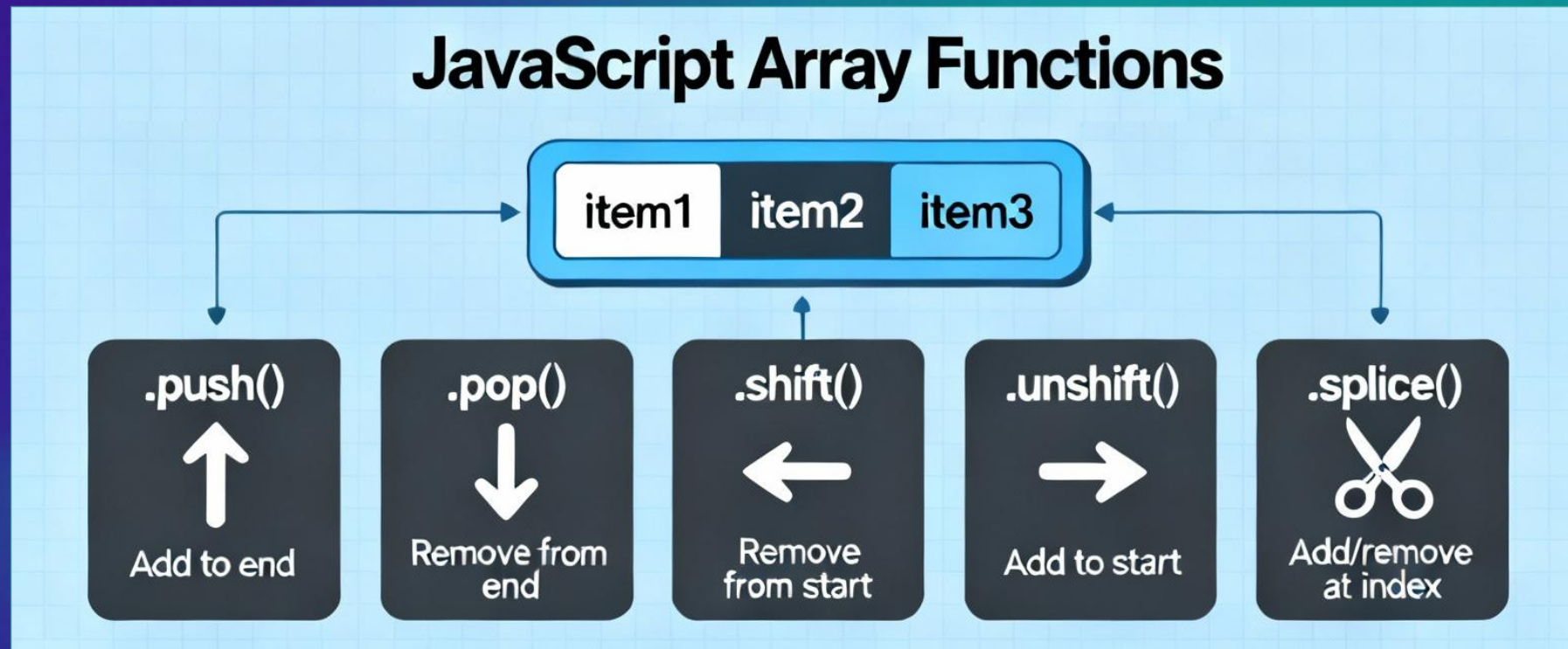
```
let x = 5, y = 10;  
console.log(x, y); // 5, 10  
[x, y] = [y, x]; // Swap x with y  
console.log(x, y); // 10, 5
```



```
function reverseArray(arr) {  
  for (let left = 0; left < arr.length/2; left++) {  
    const right = arr.length-1 - left;  
    [arr[left], arr[right]] = [arr[right], arr[left]];  
  }  
}
```

Array Operations

Add, Insert, Delete, Splice, ...



Adding Elements to Array

- **Append** a value **at the end** of array:

```
let nums = [100, 200, 300];  
nums.push(400);  
console.log(nums); // [ 100, 200, 300, 400 ]
```

- **Collect** and print numbers:

```
let nums = [];  
for (let num=1; num<=128; num*=2)  
    nums.push(num);  
console.log(nums.join(", ")); // 1, 2, 4, 8, 16, 32, 64, 128
```

Problem: Prime Numbers

- A number is **prime**, if it is divisible only by 1 and itself
 - Examples: 2, 3, 5, 19, 43, 127
- Write a JS function to find and return **all prime numbers** in given **range**:

```
console.log(findPrimeNumbers(10, 50).join(", "));  
// 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47
```

```
console.log(findPrimeNumbers(100, 150).join(", "));  
// 101, 103, 107, 109, 113, 127, 131, 137, 139, 149
```

Solution: Prime Numbers

```
function findPrimeNumbers(start, end) {  
  let primes = [];  
  for (let num = start; num <= end; num++)  
    if (isPrime(num))  
      primes.push(num);  
  return primes;  
}  
  
function isPrime(num) {  
  if (num < 2) return false;  
  for (let i = 2; i <= Math.sqrt(num); i++)  
    if (num % i == 0) return false;  
  return true;  
}
```

Prime Numbers									
1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Judge link: <https://alpha.judge.softuni.org/contests/functions-objects-events/5277>

Out of Bounds Array Operations

- Handling "**out of bounds**" array modifications:

```
let arr = [10, 20];  
arr[arr.length] = 30; // This will append 30 to the array  
console.log(arr); // [ 10, 20, 30 ]
```

```
arr[4] = 50; // This will resize the array to length = 6  
console.log(arr); // [ 10, 20, 30, <empty>, 50 ]  
console.log(arr[3]); // undefined
```

```
arr[-1] = 33; // This will add a key-value pair, not array element  
arr["maria"] = 55; // This will add a key-value pair, NOT element  
console.log(arr); // [ 10, 20, 30, <empty>, 50, '-1': 33, 'maria': 55 ]  
console.log(arr[-1]); // 33
```

Inserting Elements

- **Insert** a value **at the start** of array:

```
let nums = [200, 300];  
nums.unshift(100); // Insert at the start  
console.log(nums); // [ 100, 200, 300 ]
```

- **Insert** a value **at given index** of array:

```
let arr = ["one", "two", "three"];  
arr.splice(2, 0, "2.5"); // Insert at position 2  
console.log(arr); // [ 'one', 'two', '2.5', 'three' ]
```

Deleting Elements

- **Delete** the **last** element:

```
let fruits = ["apple", "orange", "cherry"];  
fruits.pop();  
console.log(fruits); // [ 'apple', 'orange' ]
```

- **Delete** the **first** element:

```
let fruits = ["apple", "orange", "cherry"];  
fruits.shift();  
console.log(fruits); // [ 'orange', 'cherry' ]
```

Deleting Elements at Index

- **Delete** elements at given **index**:

```
let nums = [100, 200, 300, 400, 500];  
nums.splice(1, 2);  
console.log(nums); // [ 100, 400, 500 ]
```

- **Delete + insert** elements at index (splice):

```
let nums = [100, 200, 300, 400, 500];  
nums.splice(1, 2, 333, 444, 555);  
console.log(nums); // [ 100, 333, 444, 555, 400, 500 ]
```

Extracting Sub-Arrays

- **Extract** sub-array:

```
let nums = [10, 20, 30, 40, 50, 60, 70, 80, 90];  
let subarr = nums.slice(2, 7); // Elements at index 2 ... 6  
console.log(subarr); // [ 30, 40, 50, 60, 70 ]
```

- **Combining** arrays and values into a new array:

```
let left = [20, 30, 40], right = [60, 70];  
let combined = [10, ...left, 50, ...right, 80, 90];  
console.log(combined.join('-'));  
// 10-20-30-40-50-60-70-80-90
```

Real-World Example of Arrays

- We want to build a **minimalistic editor for towns**
 - We start from these **initial towns**: [Sofia, Plovdiv, Varna]
 - Implement **add** / **edit** / **remove** a town, **move up** / **down**
- Implement this as simple **HTML page** with **JS** code
 - Keep the towns in an **array** and visualize it in a **list-box**
 - Implement all operations with buttons, which **modify the array**, and **visualize it** after that
 - Display a **success** / **error message** after each operation
- Apply simple **CSS styling** for a better look and feel

Towns Editor (with HTML and Arrays)

CHAT

 town-editor.html X

We want to build a minimalistic editor for towns

We start from these initial towns: [Sofia, Plovdiv, Varna]

Implement add / edit / remove a town, move up / down

Implement this as simple HTML page with JS code

Keep the towns in an array and visualize it in a list-box

Implement all operations with buttons, which modify the array, and visualize it after that

Display a success / error message after each operation

Apply simple CSS styling for a better look and feel

Edit ▾ Claude Haiku 4.5 ▾

 **Town Editor**

Sofia

Plovdiv

Varna

Ruse

 Add Town

 Edit Town

 Remove Town

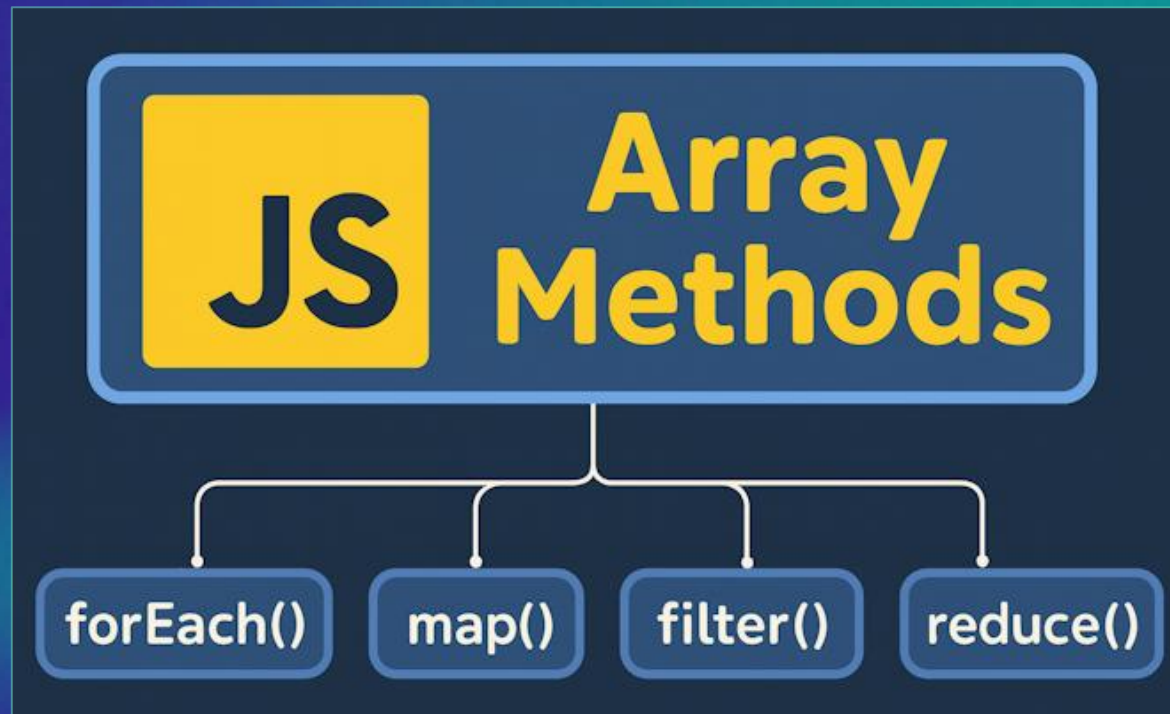
 Move Up

 Move Down

 Success: "Sofia" moved up!

Processing Arrays with Arrow Functions

Functions filter, forEach, map, reduce, ...



Filtering Array Elements

- **Filter** elements by Boolean **arrow function** (predicate):

```
console.log([1, 2, 4, 5, 6, 7, 8, 9]
  .filter(x => x % 2 == 0));
// [ 2, 4, 6, 8 ]
```

```
let coords = [{x:5,y:17}, {x:-3,y:18}, {x:7,y:-15},
  {x:-2,y:-4}, {x:0,y:6}, {x:12,y:4}, {x:3,y:8}];
let positiveCoords =
  coords.filter(p => p.x >= 0 && p.y >= 0);
console.log(positiveCoords);
// [ { x: 5, y: 17 }, { x: 0, y: 6 }, { x: 12, y: 4 }, { x: 3, y: 8 } ]
```

Transforming Elements with map()

- **Transform** each array element with **map(...)**:

```
console.log([1, 2, 3, 4, 5].map(x => x ** 2));  
// [ 1, 4, 9, 16, 25 ]
```

```
const products = [  
  { name: 'Laptop', price: 1200 },  
  { name: 'Phone', price: 800 },  
  { name: 'Printer', price: 150 }  
];  
let names = products.map(p => p.name);  
console.log(names); // ['Laptop', 'Phone', 'Printer']
```

Transforming Objects with map()

```
const products = [  
  { name: 'Laptop', price: 1200 },  
  { name: 'Phone', price: 800 },  
  { name: 'Printer', price: 150 }  
];  
  
let vatRate = 0.20; // 20% VAT tax  
  
let addVAT = p => ({ name: p.name, netPrice: p.price,  
  grossPrice: p.price * (1+vatRate) });  
  
let newProducts = products.map(addVAT);  
console.log(newProducts);
```

Iterating with `forEach()`

- **Iterating** over array elements with **`forEach(...)`**

```
const products = [  
  { name: 'Laptop', price: 1200 },  
  { name: 'Phone', price: 800 },  
  { name: 'Printer', price: 150 }  
];  
  
products.forEach(  
  (prod, idx) => console.log(  
    `${idx}. ${prod.name} -> price: ${prod.price}`  
  )  
);
```

Reducing Elements with reduce()

- **Combining elements** to a single value with **reduce(...)**:

```
const nums = [1, 2, 3, 4, 5];  
let sum = nums.reduce(  
  (s, num) => s + num, // Aggregate function  
  0 // Initial value  
);  
console.log("Sum =", sum); // Sum = 15  
  
let max = nums.reduce(  
  (max, num) => num > max ? num : max, nums[0]);  
console.log("Max =", max); // Max = 5
```

Reducing Elements with reduce()

- More-complex reduction for **object counting**:

```
const fruits = ['apple', 'banana', 'apple',  
  'orange', 'banana', 'apple'];  
  
let fCounts = fruits.reduce((counts, fruit) => {  
  counts[fruit] = (counts[fruit] || 0) + 1;  
  // counts[fruit] || 0 → returns 0 when counts[fruit] is undefined  
  return counts;  
}, {});  
  
console.log(fCounts); // { apple: 3, banana: 2, orange: 1 }
```

Finding Elements

- **Finding** elements using **find(matching function)**:

```
let users = [  
  { id: 1, name: 'Alice' },  
  { id: 2, name: 'Bob' },  
  { id: 3, name: 'Charlie' }  
];  
  
let user2 = users.find(u => u.id == 2);  
console.log(user2); // { id: 2, name: 'Bob' }  
  
let user99 = users.find(u => u.id == 99); // undefined  
console.log(user99 || "Not found"); // Not found
```


Using some() and every()

```
let students = [  
  { name: 'Alice', age: 22 },  
  { name: 'Bob', age: 17 },  
  { name: 'Charlie', age: 25 }  
];  
  
let hasAdult = students.some(s => s.age >= 18);  
console.log(hasAdult); // true  
  
let allAdults = students.every(s => s.age >= 18);  
console.log(allAdults); // false
```

Sorting Arrays

```
let students = [  
  { name: 'Alice', age: 22 },  
  { name: 'Bob', age: 17 },  
  { name: 'Charlie', age: 25 }  
];  
  
students.sort((a, b) => a.age - b.age);  
  
console.log("Students by age (ascending):");  
students.forEach(s => console.log(  
  `Student: ${s.name}, age: ${s.age}`));
```

Problem: Exclude Names

- Write a **JS function** that returns all **names** from given **array** that do ***not* appear** in a **second array**

```
let names = ["Alice", "Bob", "Charlie", "David", "Eve"];  
let ex = ["Bob", "Eve"];  
let filteredNames = excludeNames(names, ex);  
console.log(filteredNames); // ['Alice', 'Charlie', 'David']
```

- Sample solution:

```
function excludeNames(names, excludeList) {  
    return names.filter(name => !excludeList.includes(name));  
}
```

Judge link: <https://alpha.judge.softuni.org/contests/functions-objects-events/5277>

Arrays in the Real World

Contact Book App, Polygons on the Map

Name *

Bay Ivan

Phone

+359885556677

Email

bay.ivan@abv.bg

Add Contact

Search by name, phone,

Sort A-Z

Clear

Peter

0888 999-111

pp@yahoo.com

EditDelete

Maria

0805 777-8888

m@abv.bg

EditDelete

Problem: Contact Book App

- Implement a **"Contact Book" app** to manage contacts
 - Use minimalistic **HTML** and **JavaScript** code
 - Keep all contacts in **JS array**, e.g.

```
let contacts = [  
  {name: "Peter", phone: "0888 999-111", email: "pp@yahoo.com"},  
  {name: "Maria", phone: "0805 777-8888", email: "m@abv.bg"}, ... ] ;
```

- Implement **[Add]** / **[Edit]** / **[Delete]** contact, **[Sort by Name]**, **[Filter by Text]**
- Add simple **CSS styling** for better look and feel

Solution: Contact Book App

 **Contact Book**
3 contacts

Name *

Bay Ivan

Phone

+359885556677

Email

bay.ivan@abv.bg

Add Contact

 Search by name, phone,

Sort A-Z

Clear

Peter

 0888 999-111
 pp@yahoo.com

EditDelete

Maria

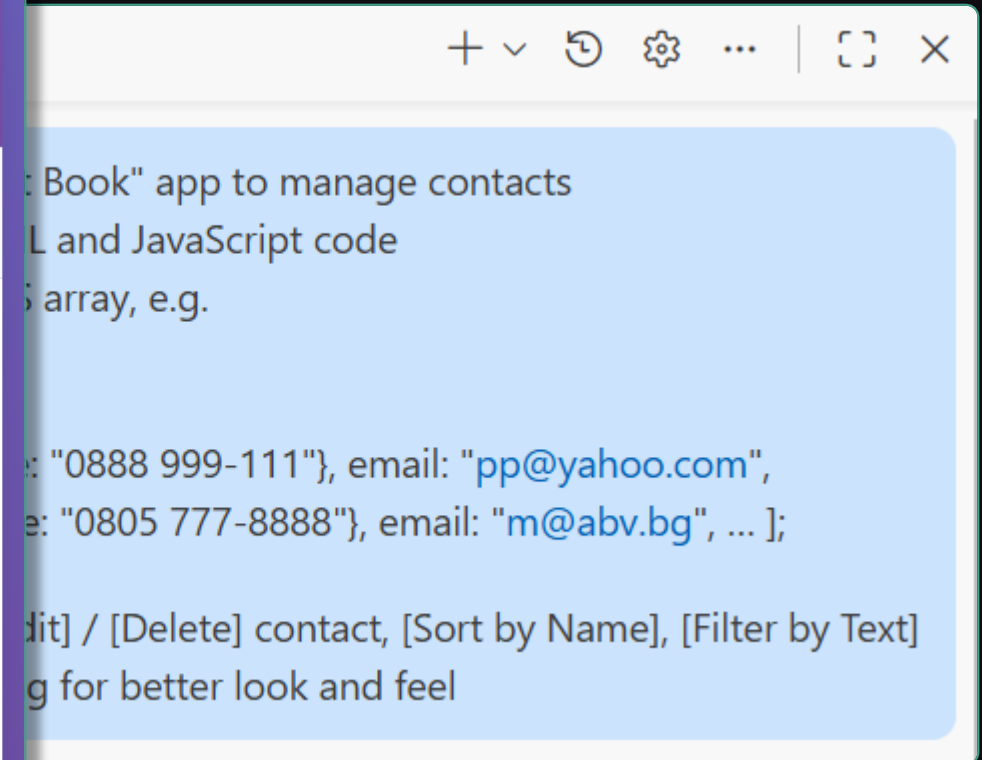
 0805 777-8888
 m@abv.bg

EditDelete

John Smith

 0889 123-456
 john@example.com

EditDelete



The screenshot shows a web browser window with a toolbar at the top containing icons for zooming, navigation, and window management. The main content area displays the Contact Book app interface, which includes a form for adding contacts and a list of existing contacts. The code editor on the right shows JavaScript code for managing contacts, including a function to add a contact and a function to delete a contact.

```
const contacts = [
  { name: "Peter", phone: "0888 999-111", email: "pp@yahoo.com" },
  { name: "Maria", phone: "0805 777-8888", email: "m@abv.bg", ... };
];

function addContact(name, phone, email) {
  contacts.push({ name, phone, email });
}

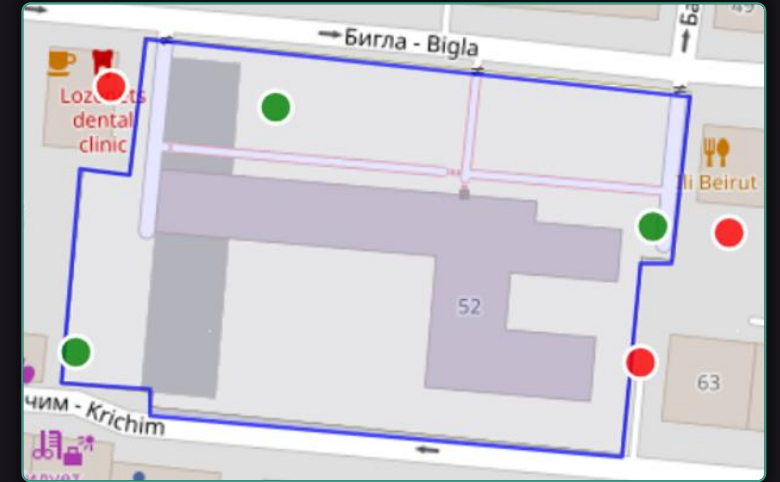
function deleteContact(name) {
  contacts = contacts.filter(contact => contact.name !== name);
}

// [Add / Edit / Delete] / [Delete] contact, [Sort by Name], [Filter by Text]
// for better look and feel
```

41

Problem: Points inside Polygon

- We want to find all **trees** on the map inside a **schoolyard**
 - We have the **coordinates of all trees**
 - We have the **schoolyard coordinates**
- Write a JS function to find all **points** inside given **polygon**



```
let trees = [ {lat:...,lng:...}, {lat:...,lng:...}, ... ];  
let schoolyard = [{lat:...,lng:...}, {lat:...,lng:...}, ... ];  
findPointsInPolygon(  
    trees, schoolyard, pointInsideCallback);
```

Problem: Points inside Polygon

```
let trees = [  
  {lat:42.67079,lng:23.32921}, {lat:42.67075,lng:23.32964},  
  {lat:42.67051,lng:23.33082}, {lat:42.67028,lng:23.32912},  
  {lat:42.67026,lng:23.33059}, {lat:42.67052,lng:23.33062}];  
  
let schoolyard = [  
  {lat:42.67088,lng:23.32930}, {lat:42.67077,lng:23.33072},  
  {lat:42.67045,lng:23.33067}, {lat:42.67045,lng:23.33059},  
  {lat:42.67008,lng:23.33054}, {lat:42.67016,lng:23.32931},  
  {lat:42.67021,lng:23.32931}, {lat:42.67022,lng:23.32908},  
  {lat:42.67063,lng:23.32913}, {lat:42.67062,lng:23.32926}];  
  
console.log("Trees inside the schoolyard:");  
findPointsInPolygon(trees, schoolyard, p => console.log(p));
```


Solutions: Points inside Polygon

```
function findPointsInPolygon(  
    trees, schoolyard, pointInsideCallback) {  
    trees  
        .filter(tree => isPointInPolygon(tree, schoolyard))  
        .forEach(tree => pointInsideCallback(tree));  
}
```

```
function isPointInPolygon(point, polygon) {  
    // TODO: implement the "ray casting" algorithm (with AI prompt)  
}
```

Judge link: <https://alpha.judge.softuni.org/contests/functions-objects-events/5277>

Visualization on the Map

 JS tress-in-schoolyard.js

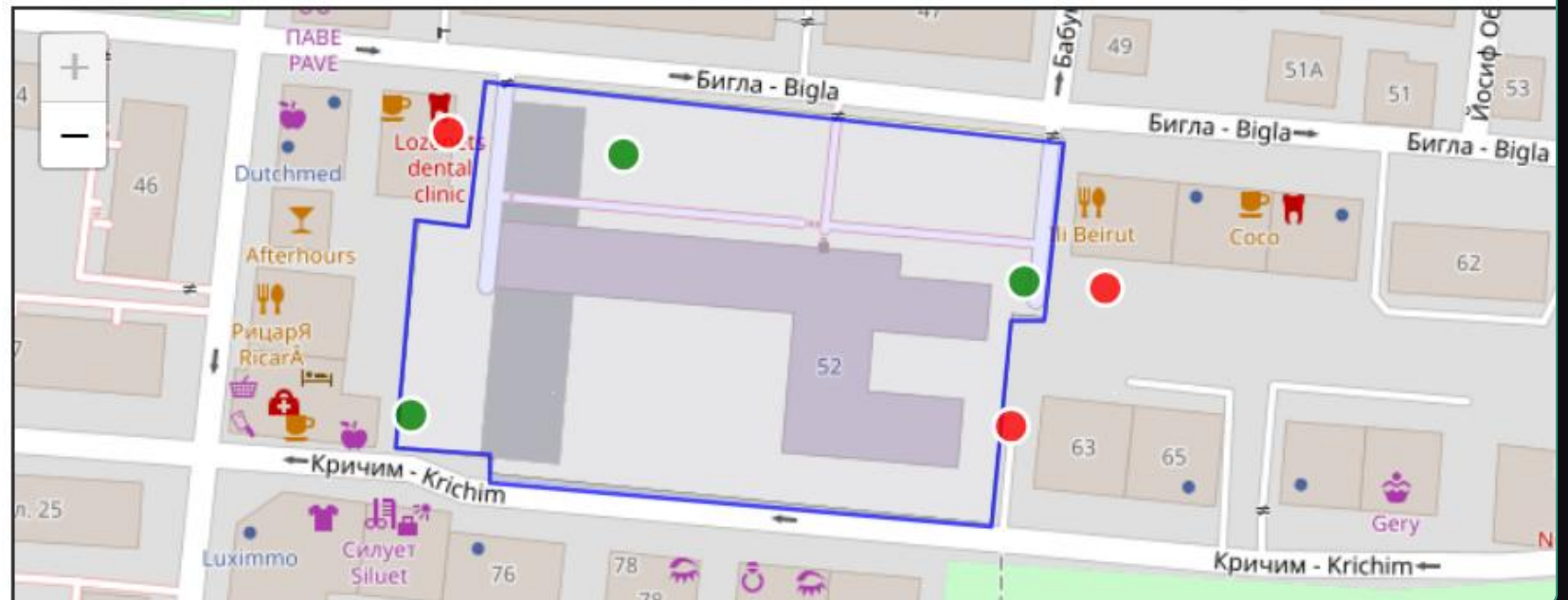
Make HTML visualization

Edit ▾ Claude Haiku 4.5 ▾

file:///C:/Projects/Arrays-Strings-Maps/tress-in-schoolyard.html

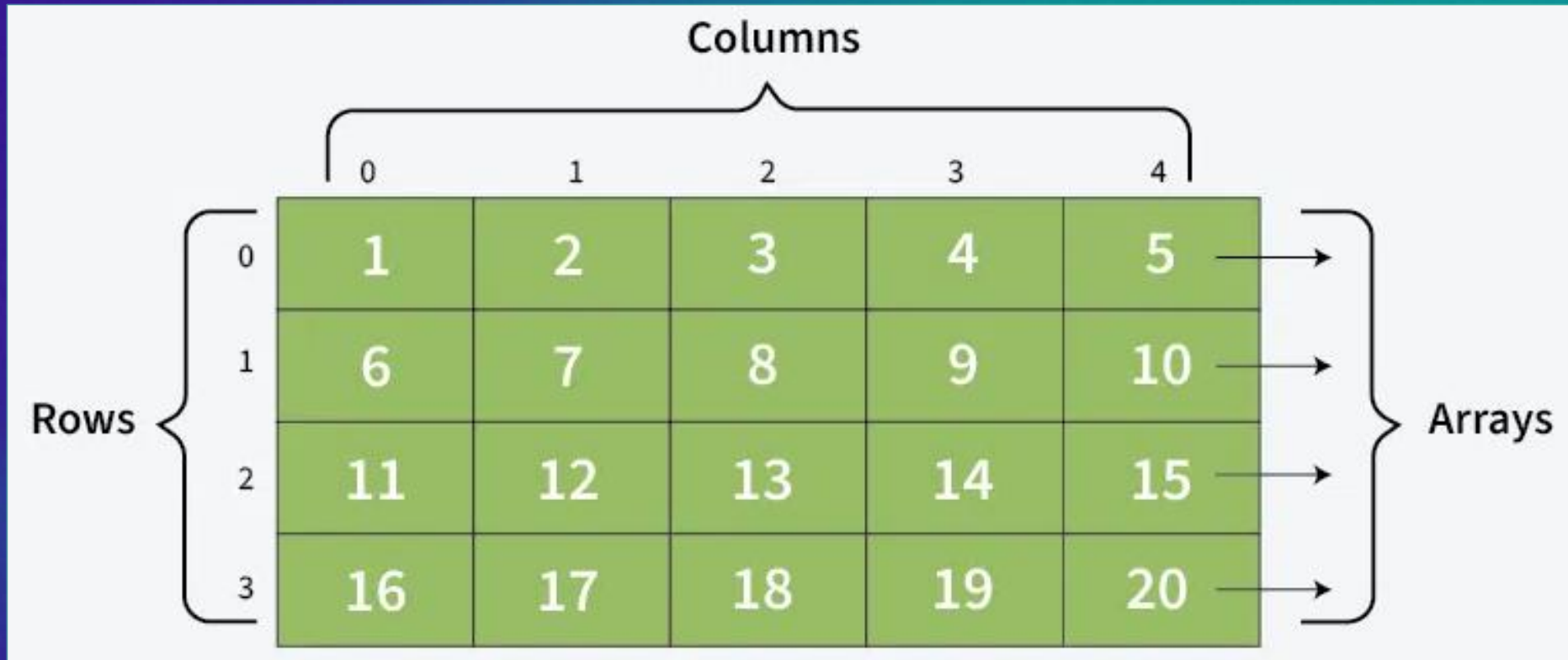
Trees in Schoolyard

 Trees inside schoolyard  Trees outside schoolyard  Schoolyard boundary



Nested Arrays

Arrays of Arrays and Matrices

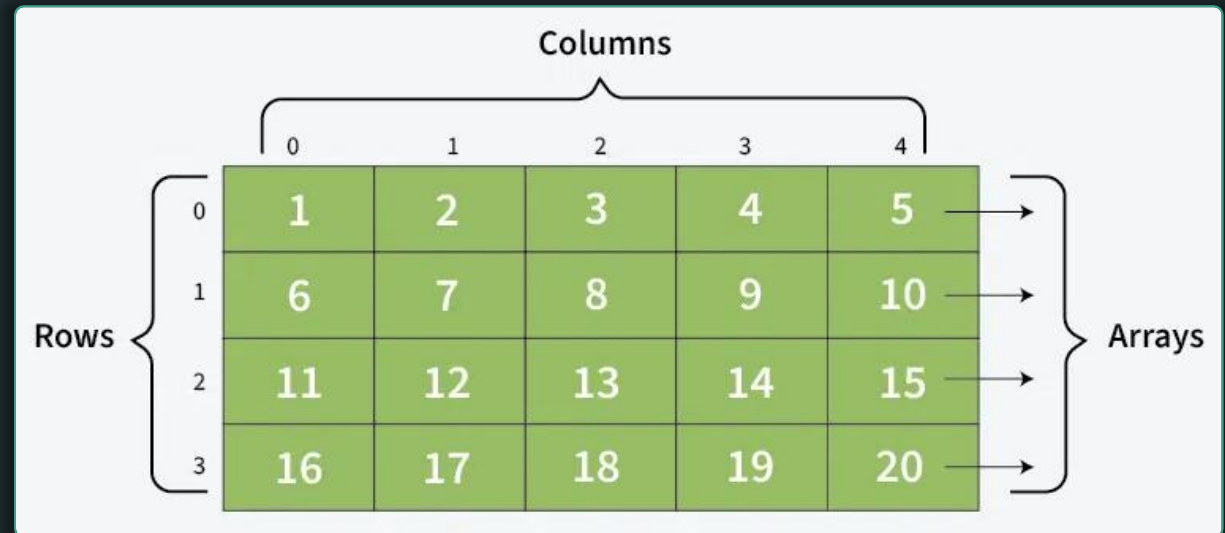


Nested Arrays

- Arrays in JS can be **nested** (array of arrays):

// A 2D matrix (array of arrays), size 4 x 5

```
let matrix = [  
  [1, 2, 3, 4, 5],  
  [6, 7, 8, 9, 10],  
  [11, 12, 13, 14, 15],  
  [16, 17, 18, 19, 20]  
];
```



```
console.log(matrix[0]); // row 0 → [1, 2, 3, 4, 5]  
console.log(matrix[1][3]); // cell {1, 3} → value 9
```

Problem: Calculating Prices

- We are given a **price list** (by **product** and **location**), e.g.

Location	coffee	water	sweets	peanuts	juice
Sofia	0.50	0.80	1.45	1.60	2.50
Plovdiv	0.40	0.70	1.30	1.50	2.40
Varna	0.45	0.70	1.35	1.55	2.35

- We have an **order**, e.g. [["coffee", 2], ["juice", 1], ["sweets", 3]]
- Write a JS function to calculate the **total price** for given **products**, **price list**, **location**, and an **order**

Example: Calculating Prices

```
const products =  
  ["coffee", "water", "sweets", "peanuts", "juice"];  
const prices = [  
  [ "Sofia", 0.50, 0.80, 1.45, 1.60, 2.50 ],  
  [ "Plovdiv", 0.40, 0.70, 1.30, 1.50, 2.40 ],  
  [ "Varna", 0.45, 0.70, 1.35, 1.55, 2.35 ] ];  
const order = [ ["coffee", 2], ["juice", 1], ["sweets", 3] ];  
console.log(calcPrices(products, prices, "Sofia", order));
```

Added 2 x coffee at 0.50 each. Subtotal: 1.00

Added 1 x juice at 2.50 each. Subtotal: 2.50

Added 3 x sweets at 1.45 each. Subtotal: 4.35

Total price for order in Sofia: 7.85

Solution: Calculating Prices

```
function calcPrices(products, priceList, location, order) {  
  let results = [], total = 0;  
  const locationPrices = priceList.find(loc => priceList[0] == loc);  
  for (const [product, quantity] of order) {  
    const productIndex = products.indexOf(product) + 1;  
    const price = locationPrices[productIndex];  
    total += price * quantity;  
    results.push(`Added ${quantity} x ${product} at ${price.toFixed(2)}  
each. Subtotal: ${(price * quantity).toFixed(2)}`);  
  }  
  results.push(`Total price for order in ${location}: ${total.toFixed(2)}`);  
  return results.join('\n');  
}
```

Judge link: <https://alpha.judge.softuni.org/contests/functions-objects-events/5277>

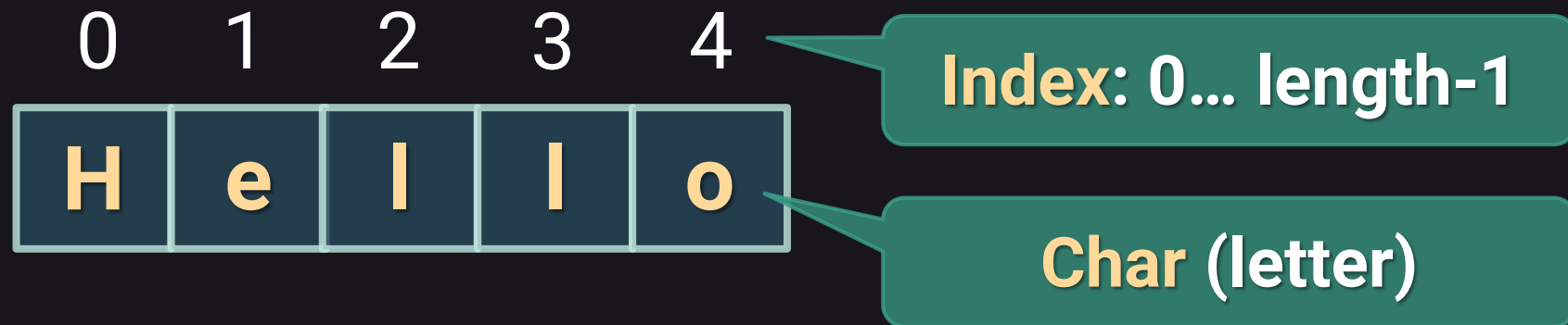
Strings and Text Processing

Strings and Basic String Operations



What is String?

- **Strings** are sequences of text characters
 - Like arrays, they have **length** and indexed access



- **Declaring** a string:

```
let greeting = "Hello", name = "JavaScript";  
let msg = greeting + ', ' + JavaScript + '!';
```

Accessing Chars by Index

- Accessing the **chars** in the string by **index**

```
let str = "Hello, JavaScript!";  
let char = str[7];  
console.log(char); // J
```

- **Iterating** over the chars of a string:

```
let str = "Hello, JavaScript!";  
for (let i=0; i<str.length; i++)  
    console.log(`str[${i}] = ${str[i]}`);  
for (let ch of str) console.log(ch);
```

Searching in a String

- **Finding a substring** in a string with **indexOf()**:

```
let str = "Hello, JS! Coding in JS is a fun.";
let jsIndex = str.indexOf("JS");
console.log(jsIndex); // 7

let jsIndex2 = str.indexOf("JS", 8);
console.log(jsIndex2); // 21

let htmlIndex = str.indexOf("HTML");
console.log(htmlIndex); // -1 (Not Found)
```

Problem: Find Occurrences

- Write a JS function to **find all occurrences** of given substring in certain string:

```
findAllOccurrences(substr, str, foundCallback = null,  
caseSensitive = true);
```

```
findAllOccurrences("JS", "Hello, JS. Visit jsTalks();  
conf", x => console.log("Found at index:", x), false);
```

Found 'JS' at index: 7

Found 'js' at index: 17

Solution: Find Occurrences

```
function findAllOccurrences(substr, str, foundCallback, ignoreCasing = true) {  
  let s = ignoreCasing ? str.toLowerCase() : str;  
  let sub = ignoreCasing ? substr.toLowerCase() : substr;  
  let startIndex = 0, indices = [];  
  while (true) {  
    const idx = s.indexOf(sub, startIndex);  
    if (idx == -1)  
      break;  
    indices.push(idx);  
    if (foundCallback)  
      foundCallback(idx, str.substr(idx, substr.length));  
    startIndex = idx + 1; // Move by 1 to allow overlapping matches  
  }  
  return indices;  
}
```

Judge link: <https://alpha.judge.softuni.org/contests/functions-objects-events/5277>

Change Casing, Trim, Repeat

```
let str = "JavaScript";  
console.log(str.toUpperCase()); // JAVASCRIPT  
console.log(str.toLowerCase()); // javascript
```

```
let multiline = `  
  This is a multiline text:  
  -> Second line\n\n`;  
console.log(multiline); // 5 lines  
console.log(multiline.trim()); // 2 lines
```

```
console.log("-".repeat(5)); // -----  
console.log("bla".repeat(3)); // blablabla
```

Comparing Strings

```
let word1 = "banana";  
let word2 = "apple";  
  
console.log(word1 == word2); // false  
console.log(word1 > word2); // true  
console.log(word1 <= word2); // false  
  
let word3 = "Banana";  
console.log(word1 == word3); // false  
console.log(  
    word1.toLowerCase() == word3.toLowerCase()); // true
```

Escaping Characters

- Special characters like `"`, `'` and *new line* are special
 - Cannot be directly inserted in string literals
 - Can be represented by an **escaping sequence**:

Escape	Meaning	Escape	Meaning	Escape	Meaning
<code>\n</code>	new line	<code>\t</code>	tab	<code>\\</code>	backslash
<code>\"</code>	double quote	<code>\'</code>	single quote	<code>\u20AC</code>	€ char

```
console.log("Lines:\n\tLine \"one\"\n\tLine \"two\"");  
console.log("\u266F \\ \u{1F389} Party time! \u{1F370}  
/ \u{1F590}"); // #\ 🎉 Party time! 🍰 / 🖐
```


Problem: Christmas Tree

- Write a JS function to return a **Christmas tree** of size **n** (odd number)

```
console.log(  
  christmasTree(5));
```

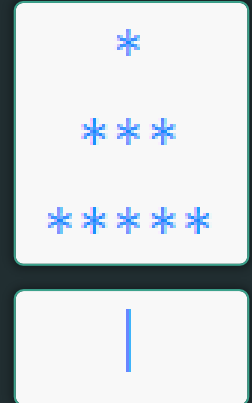
```
  *  
 ***  
*****  
  |
```

```
console.log(  
  christmasTree(7));
```

```
  *  
 ***  
*****  
*****  
  |
```

Solution: Christmas Tree

```
function christmasTree(n) {  
  let tree = '';  
  // Build the tree part  
  for (let row = 0; row < (n + 1) / 2; row++) {  
    const stars = 2 * row + 1;  
    const spaces = (n - stars) / 2;  
    tree += ' '.repeat(spaces) + '*'.repeat(stars) + '\n';  
  }  
  // Add the trunk  
  const trunkSpaces = Math.floor((n - 1) / 2);  
  tree += ' '.repeat(trunkSpaces) + '|';  
  return tree;  
}
```



More String Operations

Find, Replace, Slice, Split



Replace, Includes, Starts / Ends With

```
let str = "I love JavaScript!";  
console.log(str.replace("Java", "Type"));  
// I love TypeScript!  
console.log("banana".replaceAll("a", "o"));  
// bonono
```

```
let str = "I love JavaScript!";  
console.log(str.includes("Script")); // true  
console.log(str.includes("script")); // false
```

```
console.log("Hello".startsWith("He")); // true  
console.log("Hello".endsWith("Hi")); // false
```

Problem: Censor Words

- Write a JS function to **censor** a set of **forbidden words**
 - Take as input a **string** + array of **forbidden words**
 - Replace each word with **asterisks**, matching its **number of letters**, e.g. "Java" → "****", "C#" → "**"
 - Iterate over the chars, extract words one by one and replace each word with asterisks (don't use regex)

```
console.log(censorWords("We love coding in  
JavaScript, Python, C# and Java.", ['C#', 'Java']));
```

```
We love coding in JavaScript, Python, ** and ****.
```

Solution: Censor Words

```
function censorWords(text, forbiddenWords) {  
  let result = '', word = '';  
  for (let i=0; i < text.length; i++) {  
    let ch = text[i];  
    let nextCh = text[i+1] || '';  
    if (isAlphaNumeric(ch)) {  
      word += ch; // Part of word → append to word  
      if (!isAlphaNumeric(nextCh)) {  
        // End of word → check for censorship  
        if (forbiddenWords.includes(word)) result += '*'.repeat(word.length);  
        else result += word;  
        word = '';  
      }  
    } else result += ch; // Word separator → append to result  
  }  
  return result;  
}
```

```
let isAlphaNumeric = (char) =>  
  (char >= 'a' && char <= 'z') ||  
  (char >= 'A' && char <= 'Z') ||  
  (char >= '0' && char <= '9') ||  
  char === '#';
```

Judge link: <https://alpha.judge.softuni.org/contests/functions-objects-events/5277>

Slice and Split by Separators

```
let str = "JavaScript rules!";  
console.log(str.slice(11, 16)); // "rules" (start...end-1)  
console.log(str.slice(4)); // "Script rules!" (4...length-1)  
console.log(str.slice(-4)); // "les!" (last 4 chars)
```

```
let input = "banana, apple, orange, cherry";  
console.log(input.split(", "));  
// [ 'banana', 'apple', 'orange', 'cherry' ]
```

```
let input = "banana, apple,orange, cherry,grape";  
console.log(input.split(/,\s*/));
```


Problem: String Commands

- Write a JS function to **build a string** by executing a sequence of **commands**:
 - **Append**: {text}
 - **Prepend**: {text}
 - **Insert**: {index} {text}
 - **Delete words**: {word1}, {word2}, ... (**whole words only**)
 - **Replace**: {text} -> {text}
 - **Capitalize**: {start index} - {end index} (**inclusively**)

Example: String Commands

```
let commands = [  
    "Append: I love", "Append: , JavaScript.\n",  
    "Insert: 8 Java and ", "Insert: 8 Python and ",  
    "Insert: 8 C# and ", "Insert: 8 Golang and ",  
    "Prepend: Hello. ", "Insert: 15 TypeScript and ",  
    "Delete words: Java, Golang", "Capitalize: 9 - 12",  
    "Replace: and and -> and", "Append: Goodbye."  
];  
console.log(executeCommands(commands));
```

Hello. I LOVE, TypeScript and C# and Python and JavaScript.
Goodbye.

Solution: String Commands

- Ask the **AI assistant** to write the solution:

```
CHAT + v ↺ ⚙️ ... | [ ] ×

Write a JS function to build a string by executing a sequence of commands:
Append: {text}
Prepend: {text}
Insert: {index} {text}
Delete words: {word1}, {word2}, ... (whole words only)
Replace: {text} -> {text}
Capitalize: {start index} - {end index}

let commands = [
  "Append: I love", "Append: , JavaScript.\n",
  "Insert: 8 Java and ", "Insert: 8 Python and ",
  "Insert: 8 C# and ", "Insert: 8 Golang and ",
  "Prepend: Hello. ", "Insert: 15 TypeScript and ",
  "Delete words: Java, Golang", "Capitalize: 9 - 12",
  "Replace: and and -> and", "Append: Goodbye."
];
console.log(executeCommands(commands));
```

- Ask the **AI assistant** to make the code more **readable** and **maintainable**:

```
CHAT + v ↺ ⚙️ ... | [ ] ×

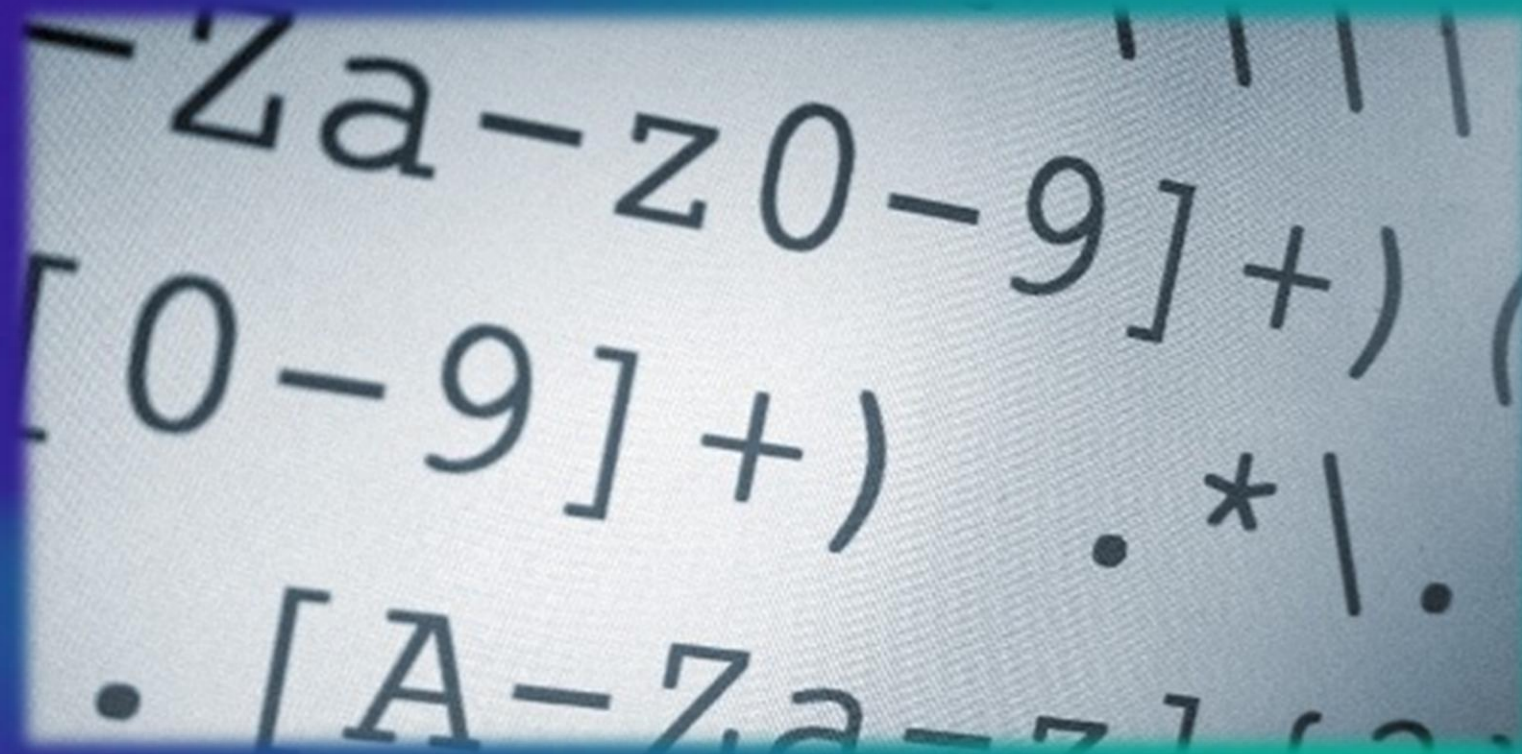
extract methods for each command

JS string-commands.js:5-29
JS string-commands.js
```

Judge link:
<https://alpha.judge.softuni.org/contests/functions-objects-events/5277>

Regular Expressions

Parsing / Editing Texts with Ease using RegExp



What Are Regular Expressions?

- A **regular expression** (RegExp) matches text by **pattern**
- **Patterns** are defined by special syntax, e.g.
 - **[0-9]+** matches non-empty sequence of digits
 - **[A-Z][a-z]*** matches a capital + small letters
 - **\b\w+\b** matches a whole word in a text
- Play with regular expressions live at:
 - <https://regexpr.com>
 - <https://regex101.com>

[←](#) [https://regex101.com](#)

☰

regular expressions 101

[@regex101](#)

[\\$ donate](#)

[📍 contact](#)

[🐛 bug reports & feedback](#)

[📖 wiki](#)

☰

REGULAR EXPRESSION

17 matches, 1035 steps (~2ms)

⋮ / ([A-Z])\w+

/ g

TEST STRING

SWITCH TO UNIT TESTS ▶

Edit the Expression & Text to see matches. Roll over matches or the expression for details. Undo mistakes with ctrl-z. Save Favorites & Share expressions with friends or the Community. Explore your results with Tools. A full Reference & Help is available in the Library, or watch the video Tutorial.

Sample text for testing:
abcdefghijklmnopqrstuvwxyz ABCDEFGHIJKLMNOPQRSTUVWXYZ
0123456789 _+-. ,!@#\$%^&*() ;\ / | < > " '
12345 -98.7 3.141 .6180 9,000 +42
555.123.4567 +1-(800)-555-2468

regex101.com

Live Demo

RegExp Patterns

- Regular expressions (**RegExp**) describe a **search pattern**
 - Used to find / extract / replace / split text by **pattern**
- Sample **pattern** and **matched texts**:

```
[A-Z][a-z]+ [A-Z][a-z]+
```

First + Last name

We are **John Smith** and **Lisa Smith**.

2 matches

Person: **Linda Davis**, age:22

1 match

I am Alex. My tel. is +163653633.

No matches

Classes and Quantifiers

Class	Matches
<code>\w</code>	word character
<code>\W</code>	non-word char
<code>\s</code>	whitespace char
<code>\S</code>	non-whitespace char
<code>\d</code>	digit [0-9]
<code>\D</code>	non-digit character
<code>\b</code>	border between words
<code>^</code>	start of text
<code>\$</code>	end of text

Pattern	Matches
<code>\w+</code>	1 or more word chars
<code>\W*</code>	0 or more non-word chars
<code>\w{2,5}</code>	2...5 word chars
<code>\s+</code>	1 or more whitespaces
<code>\d{1,3}</code>	1, 2 or 3 digits
<code>[A-Z][a-z]*</code>	word with capital first char
<code>[a-zA-Z0-9]+</code>	alpha-numeric sequence
<code>^[A-Za-z]+\$</code>	only non-empty latin letters
<code>^[0-9]+\$</code>	only non-empty digits

Regular Expressions in JS

- In JavaScript **regular expressions** are defined through a **special syntax**:

```
let regexDigits = /\d+/g;
```

```
let regexFullName = /^[a-z]+ [a-z]+$/i;
```

- Options:
 - **/i** – ignore character casing
 - **/g** – global search (find all matches)
 - **/m** – multi-line matching

Using Regular Expressions

- **Test** if a text **contains** given pattern:

```
const regex = /cat/;  
console.log(regex.test("my cat")); // true
```

- **Test** if a text **matches exactly** given pattern:

```
const regex = /^cat$/;  
console.log(regex.test("my cat")); // false
```

- **Extract** all numbers from a string:

```
console.log("Price: 20 USD, Tax: 5".match(/\d+/g));  
// ["20", "5"]
```

Using Regular Expressions (2)

- **Replace** text by given pattern:

```
console.log("Hello    world".replace(/\s+/g, " "));  
// "Hello world"
```

- **Test** if a string is a phone number like "+359 888 123456":

```
const regex = /^\\+359\\s?\\d{3}\\s?\\d{6}$/;  
console.log(regex.test("+359 888 123456")); // true
```

- **Split** by multiple delimiters:

```
console.log("one,two; three / four".split(/[\\s;\\s]/));  
// [ 'one', 'two', ' three ', ' four' ]
```

Problem: Extract Dates

- Write a JS function to **extract dates** from text
 - Valid date format: **dd-MMM-yyyy**
 - Examples: **12-Jun-1999, 3-Nov-1999**
- Example:

```
const text = "The event was on 12-Jun-1999 and another  
on 3-Nov-1999. Invalid: 32-Jan-2000, 25-abc-2020.";
console.log(extractDates(text));
```

```
[ '12-Jun-1999', '3-Nov-1999', '32-Jan-2000' ]
```

Solution: Extract Dates

```
function extractDates(text) {  
    const datePattern = /\b(\d{1,2})-(  
    (Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)-  
    (\d{4}))\b/gi;  
    const matches = text.match(datePattern);  
    return matches ? matches : [];  
}
```

Judge link: <https://alpha.judge.softuni.org/contests/functions-objects-events/5277>

Problem: Email Validation

- Write a JS function to **check if given email is valid**
 - Use a simple pattern: **user @ host...domain**
- Examples:

```
console.log(isValidEmail("user@example.com"));           // true
console.log(isValidEmail("john.doe@company.co.uk"));     // true
console.log(isValidEmail("invalid.email@"));             // false
console.log(isValidEmail("@example.com"));               // false
console.log(isValidEmail("user@domain"));               // false
console.log(isValidEmail("user.name+tag@site.org"));     // true
```

Solution: Email Validation

```
function isValidEmail(email) {  
    const emailPattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;  
    return emailPattern.test(email);  
}
```

Judge link: <https://alpha.judge.softuni.org/contests/functions-objects-events/5277>

Sets of Elements

Sets and Set Operations, Using Sets



What is a Set?

- In JavaScript, **Set** is a built-in class that stores unique values (no duplications)

```
const numbers = new Set([10, 20, 30, 30]);  
console.log(numbers); // Set { 10, 20, 30 }
```

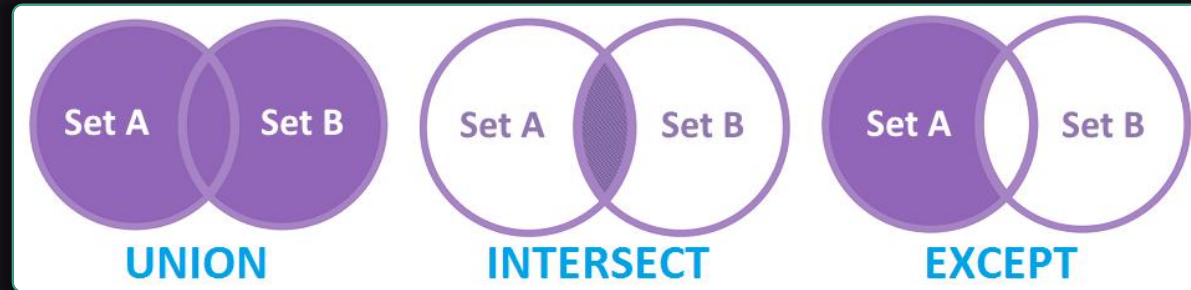
```
numbers.add(40);  
numbers.add(20); // Ignored (already exists)
```

```
numbers.delete(20);
```

```
console.log(numbers.has(30)); // true
```


Problem: Two Groups

- We are given **two groups** of students: **A** and **B** (union)



- Write a JS function to print:
 - Students who are in **both groups** (intersection)
 - Students who are **only in group A**, but not in B (A minus B)
 - Students who are **only in group B**, but not in A (B minus A)
 - All students from **both groups**, without duplicates (union)

Examples: Two Groups

```
const groupA = ["Alice", "Bob", "Charlie", "David"];  
const groupB = ["Bob", "David", "Eve", "Frank"];  
printGroupOperations(groupA, groupB);
```

Intersection A & B: ['Bob', 'David']

A minus B: ['Alice', 'Charlie']

B minus A: ['Eve', 'Frank']

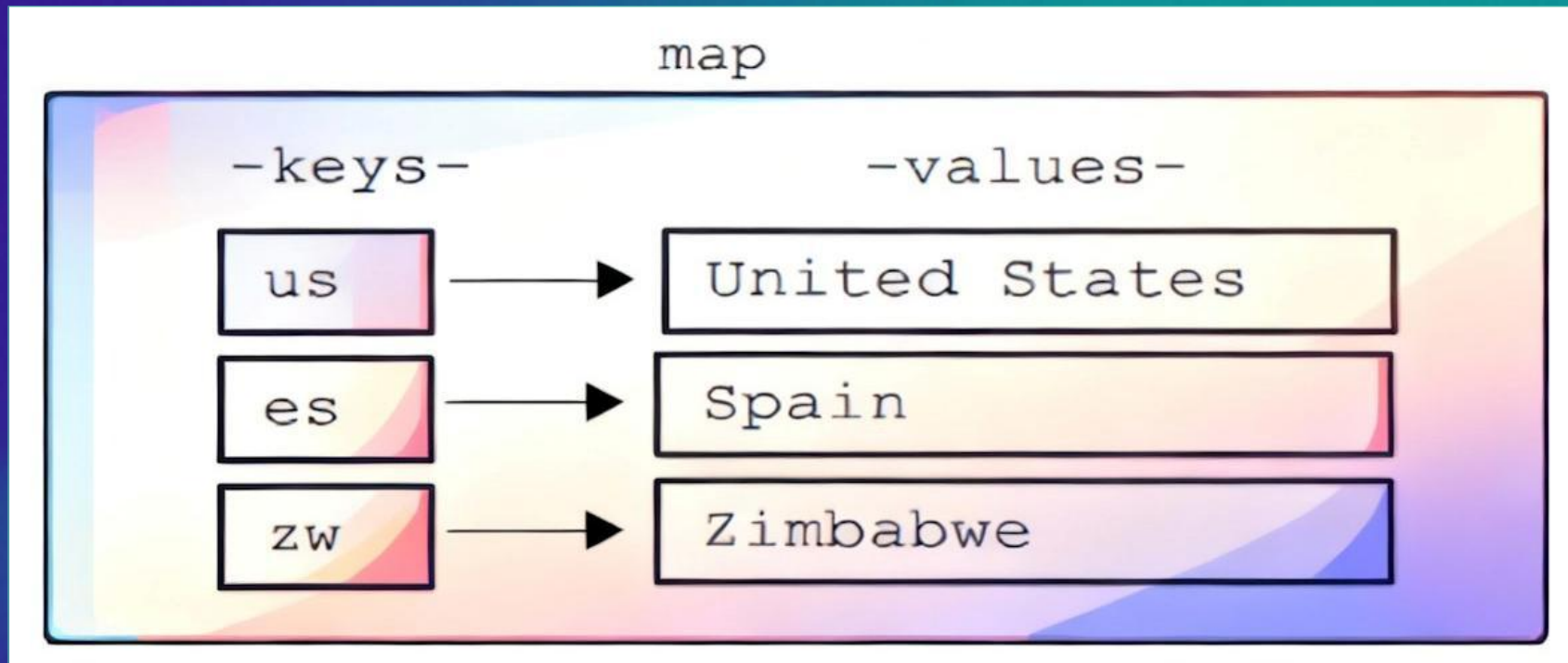
Union A | B: ['Alice', 'Bob', 'Charlie', 'David',
'Eve', 'Frank']

Solution: Two Groups

```
function printGroupOperations(groupA, groupB) {  
  const setA = new Set(groupA);  
  const setB = new Set(groupB);  
  const intersection = [...setA].filter(st => setB.has(st));  
  console.log("Intersection A & B:", intersection);  
  const onlyInA = [...setA].filter(st => !setB.has(st));  
  console.log("A minus B:", onlyInA);  
  const onlyInB = [...setB].filter(st => !setA.has(st));  
  console.log("B minus A:", onlyInB);  
  const union = [...new Set([...groupA, ...groupB])];  
  console.log("Union A | B:", union);  
}
```

Maps: Key → Value

Associative Arrays (Maps, Dictionaries)

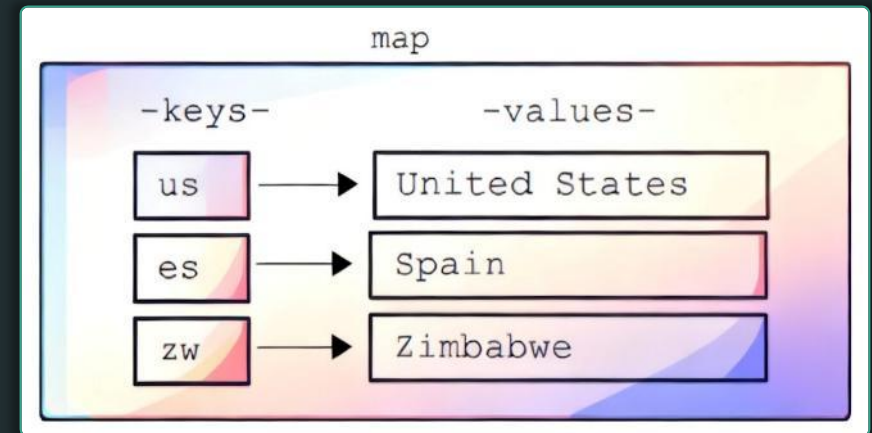


Associative Arrays (Maps)

- In programming, an **associative array** (**map**, **dictionary**) holds **key-value pairs**, where keys map to values

```
let countries = {  
  "US": "Unites States",  
  "ES": "Spain",  
  "DE": "Germany"  
};
```

key **value**



```
console.log(countries["ES"]); // Spain  
console.log(countries["XX"]); // undefined
```

Operations with Maps

- **Adding** a new **key-value** mapping / **replace** value by key:

```
countries["FR"] = "France";
```

- **Deleting** an entry **by key**:

```
delete countries["ES"];
```

- **Printing** the entire map:

```
console.log(countries);  
// { US: 'Unites States', DE: 'Germany', FR: 'France' }  
for (let code in countries) // Iterate through keys  
    console.log(code, "->", countries[code]);
```

Operations with Maps (2)

- Extracting all **keys**:

```
console.log("Keys:", Object.keys(countries));  
// Keys: [ 'US', 'ES', 'DE' ]
```

- Extracting all **values**:

```
console.log("Values:", Object.values(countries));  
// Values: [ 'Unites States', 'Spain', 'Germany' ]
```

- Iterating over the **key-value pairs**:

```
for (const [key, val] of Object.entries(countries))  
  console.log(`${key} -> ${val}`);
```

Problem: Counting Occurrences

- Write a JS function that takes as an input an **array** and counts **how many times** each array elements occurs

```
const animals = ['dog', 'cat', 'cat', 'dog', 'cat', 'fox'];  
console.log(countOccurrences(animals));  
// { dog: 2, cat: 3, fox: 1 }
```

```
function countOccurrences(arr) {  
  const counts = {};  
  for (const item of arr)  
    counts[item] = (counts[item] || 0) + 1;  
  return counts;  
}
```

Judge link: <https://alpha.judge.softuni.org/contests/functions-objects-events/5277>

Problem: Shopping List

- Write a JS function to **process shopping list** items:
 - **Products needed**, e.g.: {bread:2, butter:1, yogurt:4, eggs:10}
 - **Products available**, e.g. {butter:1, eggs:3}
 - **Price list**: {butter:5.30, eggs:0.40, bread:3.50, yogurt:1.65}
- Print all **ordered items** in this format:

```
2 x bread -> 7.00
4 x yogurt -> 6.60
7 x eggs -> 2.80
Total: 16.40
```

Solution: Shopping List

```
function processShoppingList(needed, available, prices) {  
  let total = 0;  
  for (const item in needed) {  
    const availableQty = available[item] || 0;  
    const missingQty = needed[item] - availableQty;  
    if (missingQty > 0) {  
      let cost = missingQty * prices[item];  
      console.log(` ${missingQty} x ${item} -> ${cost.toFixed(2)} `);  
      total += cost;  
    }  
  }  
  console.log(` Total: ${total.toFixed(2)} `);  
}
```

Judge link: <https://alpha.judge.softuni.org/contests/functions-objects-events/5277>

Shopping List as HTML Page

- Create a simple HTML + JS app **process shopping list** items:
 - Left column: **products needed**, e.g.: {bread:2, butter:1, yogurt:4, eggs:10}. Implement add, remove
 - Mid column: **products available**, e.g. {butter:1, eggs:3}. Implement add, remove
 - Right column: **price list**, e.g. {butter:5.30, eggs:0.40, bread:3.50, yogurt:1.65}. Implement add, remove
 - Button to **calculate** and fill in a table the items to order + total price
- Keep all app data in 3 **simple key-value-pair** objects: **productsNeeded**, **productsAvailable**, **priceList**
- Implement minimalistic **CSS** for responsive layout and styling

Sample "Shopping List" App



Shopping List Manager

Products Needed

Product name

1

Add

bread

Qty: 2

Remove

butter

Qty: 1

Remove

yogurt

Qty: 4

Remove

eggs

Qty: 10

Remove

Products Available

Product name

1

Add

butter

Qty: 1

Remove

eggs

Qty: 3

Remove

Price List

Product name

0.00

Add

butter

\$5.30

Remove

eggs

\$0.40

Remove

bread

\$3.50

Remove

yogurt

\$1.65

Remove

Calculate Order & Total

Order Summary

Product	Needed	Available	To Order	Unit Price	Total Price
bread	2	0	2	\$3.50	\$7.00
butter	1	1	0	\$5.30	\$0.00
yogurt	4	0	4	\$1.65	\$6.60
eggs	10	3	7	\$0.40	\$2.80

Total Cost: \$16.40

0}. Implement add, remove
d, remove
:1.65}. Implement add, remove
, productsAvailable, pricelist

<>

shopping.html

95

Problem: Shop

- You are given a **shop data** (a nested structure) like this:

```
let shop = {  
  products: ['bread', 'butter', 'eggs', 'yogurt'],  
  prices: { butter:5.30, eggs:0.40, bread:3.50, yogurt:1.65 },  
  locations: [  
    { name:"Sofia", lat:42.657, lng:23.316, sales: [ ['bread',2],  
      ['eggs',5], ['butter',1], ['bread',2], ['yogurt',2] ]},  
    { name:"Plovdiv", lat:42.145, lng:24.779, sales: [ ['eggs',3],  
      ['butter',2], ['bread',1], ['yogurt',4], ['bread',3] ] }  
  ]  
}
```

- Calculate the **total incomes** by **location** and **product**

Solution: Shop

```
function calculateIncomes(shop) {  
  const incomeByLocation = {};  
  shop.locations.forEach(location => {  
    incomeByLocation[location.name] = {};  
    location.sales.forEach(([product, quantity]) => {  
      const price = shop.prices[product];  
      const income = price * quantity;  
      incomeByLocation[location.name][product] =  
        (incomeByLocation[location.name][product] || 0) + income;  
    });  
  });  
  return incomeByLocation;  
}
```

Judge link: <https://alpha.judge.softuni.org/contests/functions-objects-events/5277>

Lesson Summary

- **Arrays** hold sequences of elements, e.g. **[20, 30, 40, 50]**
 - Operations: length, access by index [], push, pop, shift, unshift, slice, splice, forEach, filter, map, reduce, ...
- **Strings** hold text, e.g. **"some text"**
 - Operations: length, get by index [], indexOf, toLowerCase, repeat, trim, replace, startsWith, includes, slice, RegExp
- **Sets** hold unique value (no duplicates): **new Set([10, 20])**
- **Maps** hold key-value pairs: **{ "bread":3, "butter":1 }**
 - Operations: access by index [], keys, values, entries



Questions?



Postbank – Exclusive Partner for SoftUni AI



- One of the leading **banking institutions** in Bulgaria
- Member of the Eurobank Group with € 99.6 billion of assets
- Innovative trendsetter in the digitalization and transformation
- Certified Top Employer 2024 by the international Top Employers Institute
- AI integration for business, learning and development
- AI Assistants for talents: Story Builder, CV Assistant, Interview Trainer
- Proven people care and wellbeing initiatives
- Benefits and unlimited access to professional, personal and leadership trainings and programs
- www.postbank.bg / careers.postbank.bg



Diamond Partners of Software University



**SUPER
HOSTING
.BG**



THE CROWN IS YOURS



VIVACOM

Diamond Partners of SoftUni Digital



Diamond Partners of SoftUni Digital



HUMAN

IMPULSE MEDIA®



1FORFIT

NETPEAK
DIGITAL GROWTH PARTNER

ABC DESIGN &
COMMUNICATION

Zahara
dig.it.all

ETIENYANEV
Break Your *Limits*. Live Your *Brand*.



Diamond Partners of SoftUni Creative



**SUPER
HOSTING
.BG**



Organization Partners of SoftUni Creative



dequitas.

Фигмаўстор



Studio π

Think Big • Design Smart

дизайнът
на нещата

REDV[®] PRINT
STUDIO

THE
BUCKS
TOWN'S
WORK